



UNIVERSIDAD NACIONAL AGRARIA

LA MOLINA

Dpto. académico de Estadística e Informática





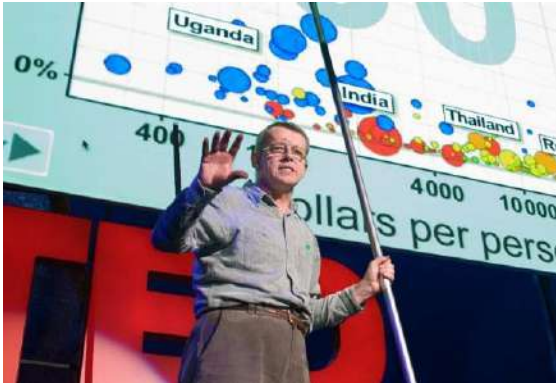
Mg. Jesús Salinas Flores

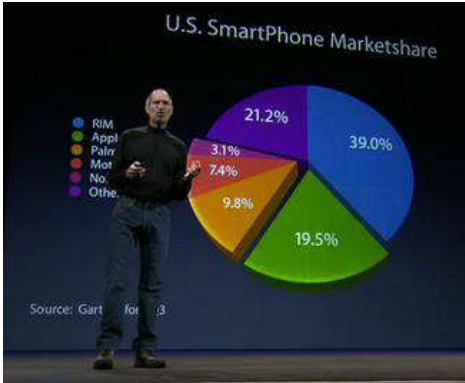
jsalinas@lamolina.edu.pe

Técnicas de Exploración de Datos



Representantes del Data StoryTelling





Manipulación de datos con dplyr



Mg. Jesús Salinas Flores



jsalinas@lamolina.edu.pe

Librería magrittr



1. Reducir el tiempo de desarrollo.
2. Mejorar la legibilidad y el mantenimiento del código.

Fuente: https://www.uv.es/vcoll/curso_r.html

%>%
magrittr

Ceci n'est pas un pipe.



Operador pipe (%>%)

- ❖ Este operador permite encadenar llamadas a funciones para así realizar de forma sencilla transformaciones de datos complejas.
- ❖ El **operador pipe** se lo debemos a **Stefan Bache** en su paquete **magrittr**.
- ❖ En resumen, lo que hace este operador es **pasar el elemento que está a su izquierda como un argumento de la función que tiene a la derecha**.

`función(objeto, argumentos de la función)` es equivalente a: `objeto %>% función(argumentos de la función)`



Fuente: https://www.uv.es/vcoll/curso_r.html



ViewPipeSteps

```

1 iris %>%
2   select(Species, Petal.Length) %>%
3   filter(Petal.Length > 1.6) %>%
4   group_by(Species) %>%
5   summarise(plsum = sum(Petal.Length))
6

```

VIEWPIPESTEPS

src: @daranzolin



APRENDE A UTILIZAR %>% O PIPE EN R

R es un lenguaje funcional, lo que significa que su código a menudo contiene muchos paréntesis (). Cuando tenga un código complejo, esto a menudo significa anidar entre muchos paréntesis. Esto hace que su código R sea difícil de leer y comprender. ¡Aquí es donde %>% entra al rescate!

ECHA UN VISTAZO AL SIGUIENTE EJEMPLO

Filtraremos las filas donde 'zona' sea igual 'sur' utilizando 2 métodos, uno de ellos con el método %>%.

METODO 1

```
filter(datos, zona == 'Sur')
```

Datos Condición de filtrado

METODO 2

```
datos %>% filter(zona == 'Sur')
```

Datos Pipe Condición de filtrado

CONCLUSIONES

Para expresar una secuencia de múltiples operaciones, puede utilizar la tubería %>%. Esto toma el resultado de una función o expresión y lo pasa a otra función como argumento.

PIPE ADICIONALES Y DOBLES

Filtraremos las columnas 'ingreso' y 'zona' donde 'zona' sea igual 'sur'.

```
datos %>% select(ingreso, zona) %>% filter(zona == 'Sur')
```

Pipe



Data School Bolivia

SIGUENOS





ESTADÍSTICA PARA TODOS



10 TAREAS DE MANIPULACION DE DATOS CON DPLYR



Resumir múltiples columnas

Contar el número de casos o filas únicas (con NAs)

Contar el número de casos de filas

Suma a través de filas / columnas

Seleccionar casos completos

Cambiar el nombre de las variables

Eliminar filas duplicadas

Frecuencia / proporciones relativas

Aplicar una función a cada fila de una tabla

Seleccionar filas con los valores máximos en cada grupo



Data School Bolivia

SIGUENOS:





ESTADÍSTICA PARA TODOS



Funciones del dplyr

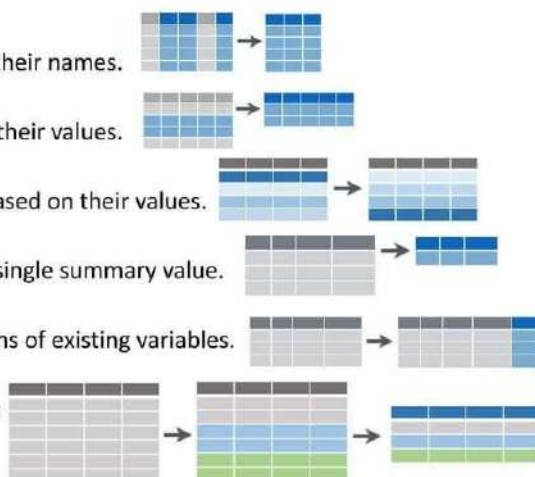
- **filter():** filtra filas que cumplen con el criterio
- **arrange():** ordena los datos por columna columna en orden ascendente o descendente
- **mutate():** crea nuevas columnas conservando las variables existentes
- **select():** selecciona columnas de los datos
- **group_by ():** agrupa diferentes observaciones
- **summarise():** resume cualquiera de las funciones anteriores
- **count():** cuenta observaciones
- **join():** realiza left, right, full y inner join en R



Key Functions

For more, see this great resource: [Data Wrangling Cheat Sheet](#)

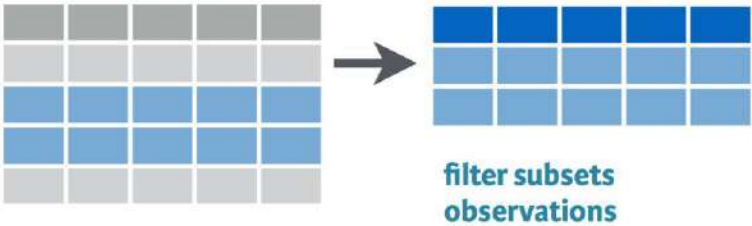
- **select()**
 - Picks variables (columns) based on their names.
- **filter()**
 - Picks observations (rows) based on their values.
- **arrange()**
 - Changes the ordering of the rows based on their values.
- **summarise()**
 - Reduces multiple values down to a single summary value.
- **mutate()**
 - Adds new variables that are functions of existing variables.
- **group_by()**
 - Performs data operations on groups that are defined by variables.





filter()

filter()



Fuente: <https://www.datacamp.com>



filter()

Símbolo	Significado	Símbolo	Significado
>	Mayor que	!=	distinto a
<	Menor que	%in%	dentro del grupo
==	Igual a	is.na	es NA
>=	Mayor o igual que	!is.na	no es NA
<=	Menor o igual que	&,	y , o



filter()

dplyr::filter() KEEP ROWS THAT satisfy your **CONDITIONS**

keep rows from... this data... ONLY IF... type is "otter" AND site is "bay"

```
filter(df, type == "otter" & site == "bay")
```

type	food	site
otter	urchin	bay
shark	seal	channel
otter	abalone	bay
otter	crab	wharf

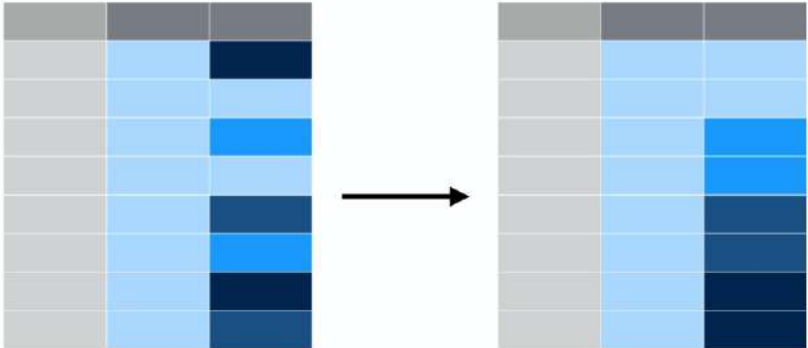
Illustration: An orange character points to a map of a bay. A purple character and a green character mark rows in the table with checkmarks and X's respectively, indicating which rows are kept or filtered out based on the conditions.

Fuente: <https://github.com/allisonhorst/stats-illustrations>



arrange()

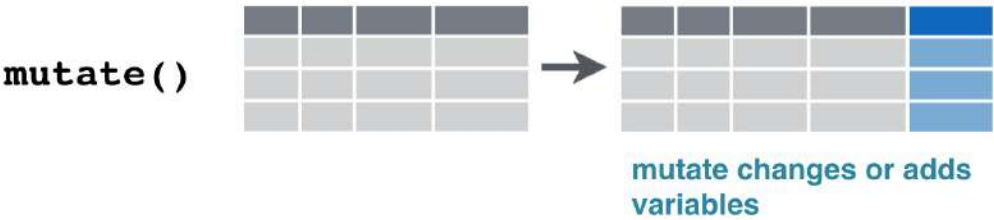
arrange() sorts a table based on a variable



Fuente: <https://www.datacamp.com>



mutate()



Fuente: <https://www.datacamp.com>



mutate()

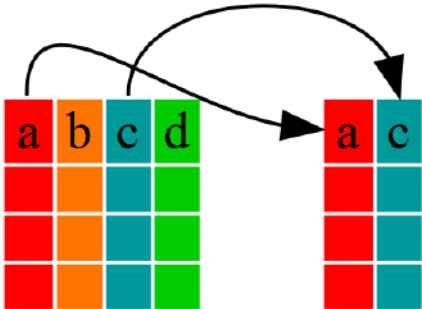


Ilustración realizada por Allison Horst. Fuente: <https://blog.rstudio.com/2019/11/18/artist-in-residence/>



select()

```
select(data.frame,a,c)
```

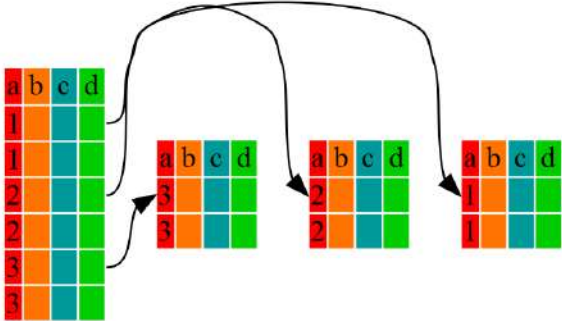


Fuente: <https://swcarpentry.github.io/r-novice-gapminder-es/13-dplyr/>



group_by()

```
gapminder %>% group_by(a)
```



Fuente: <https://swcarpentry.github.io/r-novice-gapminder-es/13-dplyr/>



summarize()

summarize() turns many rows into one

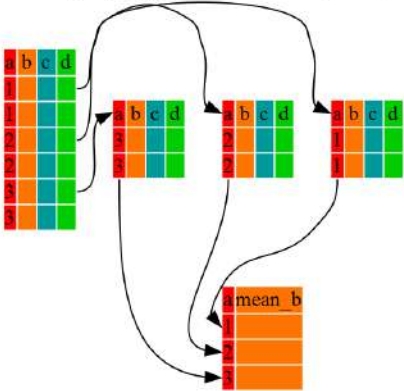


Fuente: <https://www.datacamp.com>

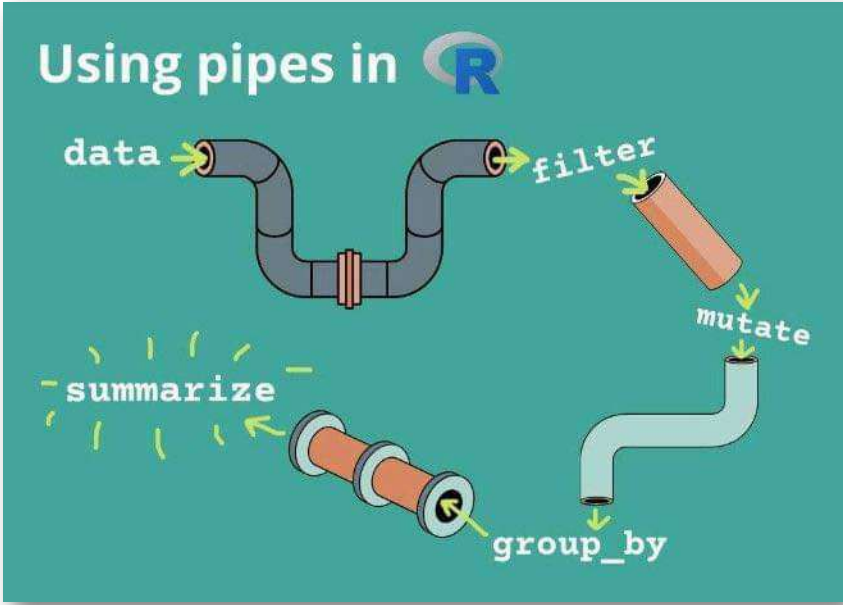


summarize()

```
gapminder %>% group_by(a) %>% summarize(mean_b=mean(b))
```



Fuente: <https://swcarpentry.github.io/r-novice-gapminder-es/13-dplyr/>



3. Organiza tus datos con *arrange()* en función de una o más columnas, de forma ascendente o descendente.



4. Enriquece tus datos con *mutate()*, generando nuevas columnas basadas en cálculos o transformaciones.



5. Utiliza *summarise()* para agrupar y obtener estadísticas clave o métricas agregadas de tus datos.



1. Usa *select()* para simplificar tus datos seleccionando solo las columnas que necesitas.

2. Aplica *filter()* para extraer subconjuntos de tus datos basándote en condiciones específicas.

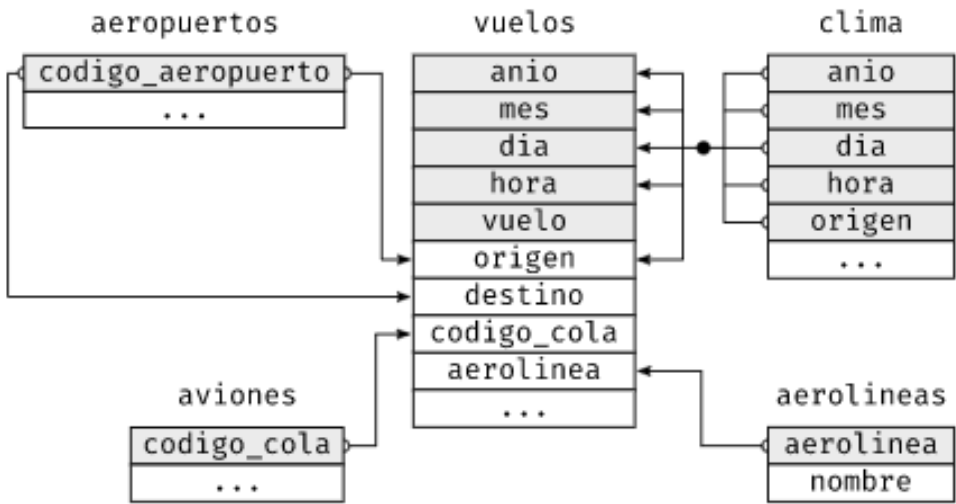


Datos relacionales con dplyr

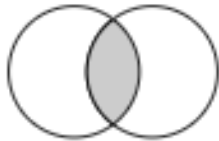


Mg. Jesús Salinas Flores

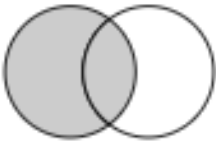
jsalinas@lamolina.edu.pe



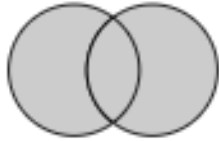
Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



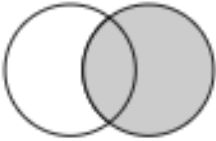
Unión interior
`inner_join(x,y)`



Unión por la izquierda
`left_join(x,y)`

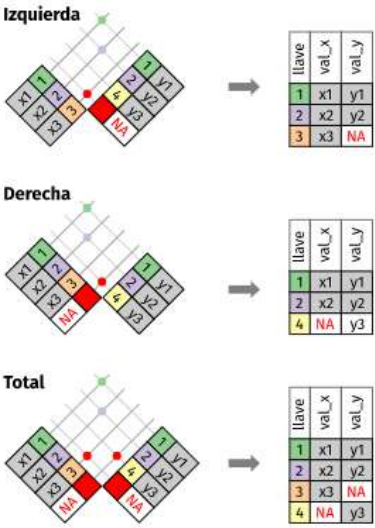


Unión total
`full_join(x,y)`



Unión por la derecha
`right_join(x,y)`

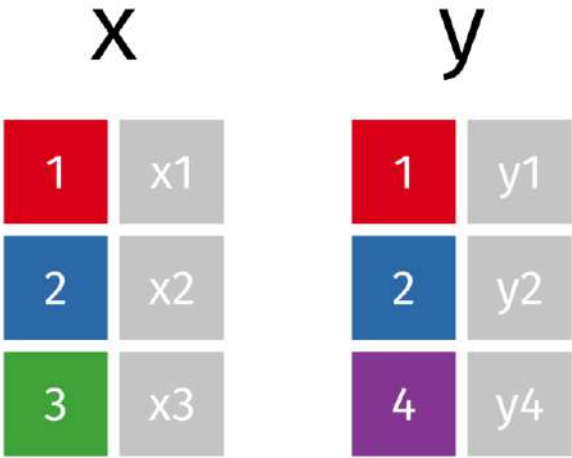
Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



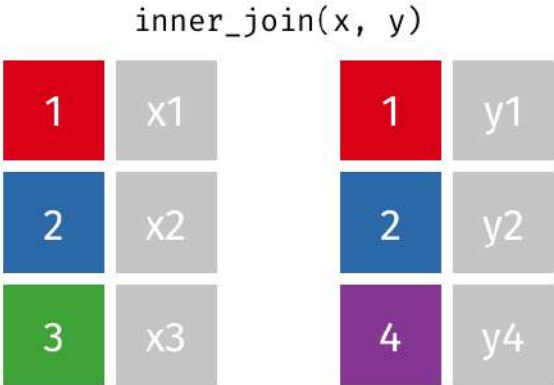
```
x
#> # A tibble: 3 x 2
#>   id x
#>   <int> <chr>
#> 1     1 x1
#> 2     2 x2
#> 3     3 x3
y
#> # A tibble: 3 x 2
#>   id y
#>   <int> <chr>
#> 1     1 y1
#> 2     2 y2
#> 3     4 y4
```



Fuente: <https://github.com/gadenbuie/tidyexplain>



```
inner_join(x, y, by = "id")
#> # A tibble: 2 x 3
#>   id x    y
#>   <int> <chr> <chr>
#> 1     1 x1   y1
#> 2     2 x2   y2
```

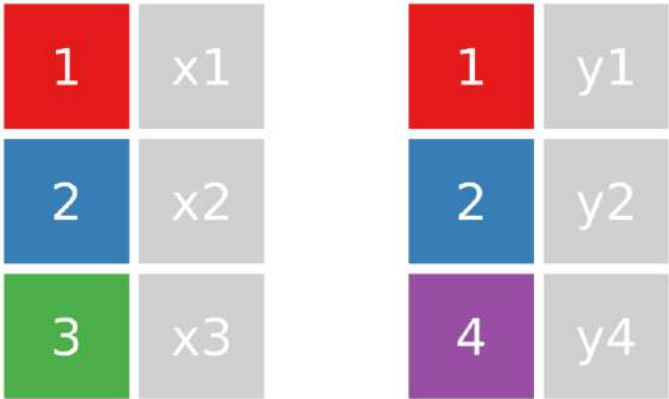


Fuente: <https://github.com/gadenbuie/tidyexplain>



left_join(x, y)

```
x
#> # A tibble: 3 x 2
#>   id x
#>   <int> <chr>
#> 1     1 x1
#> 2     2 x2
#> 3     3 x3
y
#> # A tibble: 3 x 2
#>   id y
#>   <int> <chr>
#> 1     1 y1
#> 2     2 y2
#> 3     4 y4
```

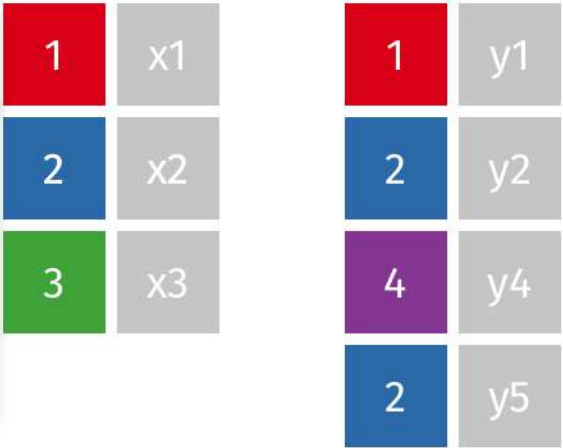


Fuente: <https://github.com/gadenbuie/tidyexplain>



left_join(x, y)

```
y_extra # has multiple rows with the key from `x`
#> # A tibble: 4 x 2
#>   id y
#>   <dbl> <chr>
#> 1     1 y1
#> 2     2 y2
#> 3     4 y4
#> 4     2 y5
left_join(x, y_extra, by = "id")
#> # A tibble: 4 x 3
#>   id x    y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
#> 2     2 x2    y2
#> 3     2 x2    y5
#> 4     3 x3    NA
```



Fuente: <https://github.com/gadenbuie/tidyexplain>



right_join(x, y)

```
right_join(x, y, by = "id")
#> # A tibble: 3 x 3
#>   id x    y
#>   <int> <chr> <chr>
#> 1     1 x1   y1
#> 2     2 x2   y2
#> 3     4 <NA> y4
```

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Fuente: <https://github.com/gadenbuie/tidyexplain>



full_join(x, y)

```
full_join(x, y, by = "id")
#> # A tibble: 4 x 3
#>   id x    y
#>   <int> <chr> <chr>
#> 1     1 x1   y1
#> 2     2 x2   y2
#> 3     3 x3   <NA>
#> 4     4 <NA> y4
```

1	x1	1	y1
2	x2	2	y2
3	x3		
		4	y4

Fuente: <https://github.com/gadenbuie/tidyexplain>



semi_join(x, y)

```
semi_join(x, y, by = "id")
#> # A tibble: 2 x 2
#>   id x
#>   <int> <chr>
#> 1     1 x1
#> 2     2 x2
```

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Fuente: <https://github.com/gadenbuie/tidyexplain>



anti_join(x, y)

```
anti_join(x, y, by = "id")
#> # A tibble: 1 x 2
#>   id x
#>   <int> <chr>
#> 1     3 x3
```

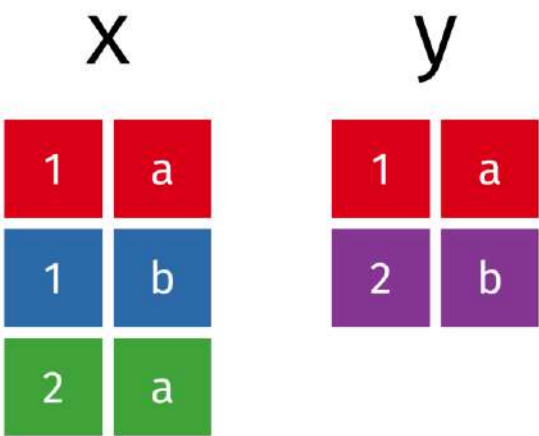
1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Fuente: <https://github.com/gadenbuie/tidyexplain>



```
x
#> # A tibble: 3 x 2
#>   x     y
#>   <chr> <chr>
#> 1 1     a
#> 2 1     b
#> 3 2     a

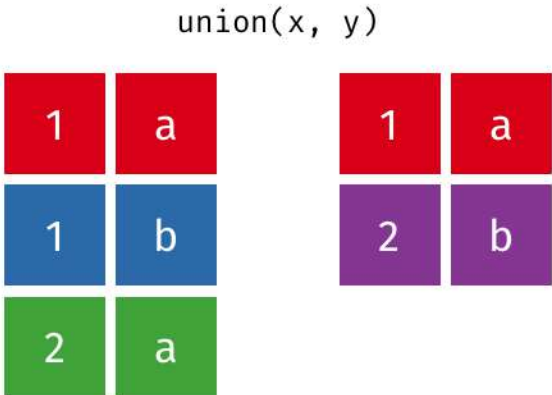
y
#> # A tibble: 2 x 2
#>   x     y
#>   <chr> <chr>
#> 1 1     a
#> 2 2     b
```



Fuente: <https://github.com/gadenbuie/tidyexplain>



```
union(x, y)
#> # A tibble: 4 x 2
#>   x     y
#>   <chr> <chr>
#> 1 2     b
#> 2 2     a
#> 3 1     b
#> 4 1     a
```

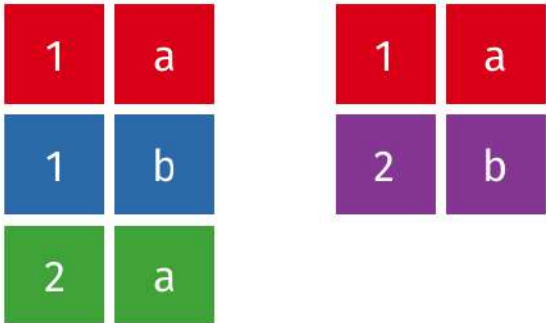


Fuente: <https://github.com/gadenbuie/tidyexplain>



union(y, x)

```
union(y, x)
#> # A tibble: 4 x 2
#>   x     y
#>   <chr> <chr>
#> 1 2     a
#> 2 1     b
#> 3 2     b
#> 4 1     a
```

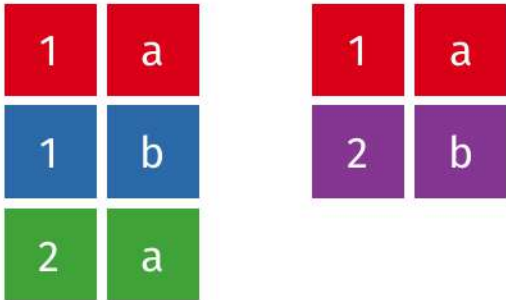


Fuente: <https://github.com/gadenbuie/tidyexplain>



union_all(x, y)

```
union_all(x, y)
#> # A tibble: 5 x 2
#>   x     y
#>   <chr> <chr>
#> 1 1     a
#> 2 1     b
#> 3 2     a
#> 4 1     a
#> 5 2     b
```

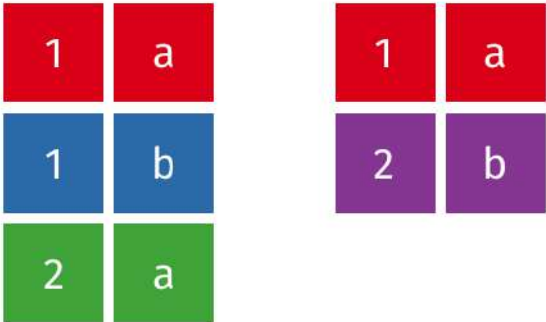


Fuente: <https://github.com/gadenbuie/tidyexplain>



intersect(x, y)

```
intersect(x, y)
#> # A tibble: 1 x 2
#>   x     y
#>   <chr> <chr>
#> 1 1     a
```

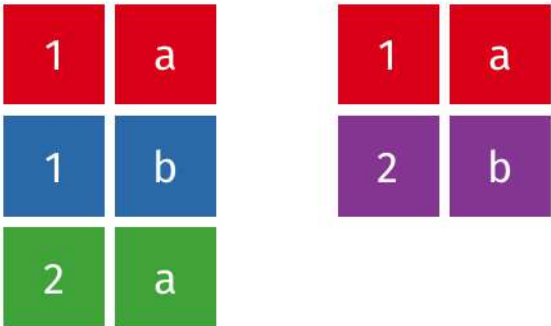


Fuente: <https://github.com/gadenbuie/tidyexplain>



setdiff(x, y)

```
setdiff(x, y)
#> # A tibble: 2 x 2
#>   x     y
#>   <chr> <chr>
#> 1 1     b
#> 2 2     a
```



Fuente: <https://github.com/gadenbuie/tidyexplain>



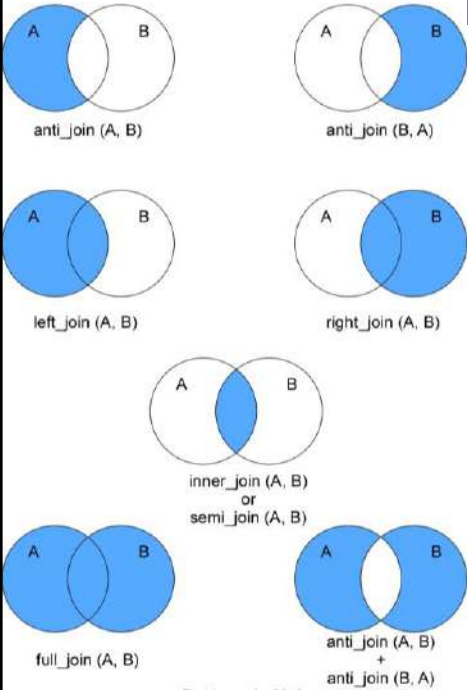
setdiff(y, x)

```
setdiff(y, x)
#> # A tibble: 1 x 2
#>   x     y
#>   <chr> <chr>
#> 1 2     b
```

1	a
1	b
2	a

1	a
2	b

Fuente: <https://github.com/gadenbuie/tidyexplain>



Fuente: https://rstudio-pubs-static.s3.amazonaws.com/369629_d807323ee988436ea6b4d4bab5647047.html

Rodrigues A., 2018

Manipulación de data frames con tidyr



Mg. Jesús Salinas Flores



jsalinas@lamolina.edu.pe

“**TIDY DATA** is a standard way of mapping the meaning of a dataset to its structure.”

—HADLEY WICKHAM

In tidy data:

- each variable forms a column
- each observation forms a row
- each cell is a single measurement

Fuente: <https://github.com/allisonhorst/stats-illustrations>

each column a variable

id	name	color
1	floof	gray
2	max	black
3	cat	orange
4	donut	gray
5	merlin	black
6	panda	calico

each row an observation

Wickham, H. (2014). Tidy Data. Journal of Statistical Software 59 (10). DOI: 10.18637/jss.v059.i10



Conjunto de datos ordenado

Existen tres reglas interrelacionadas que hacen que un conjunto de datos sea ordenado:

- 1. Cada variable debe tener su propia columna.
- 2. Cada observación debe tener su propia fila.
- 3. Cada valor debe tener su propia celda.



Conjunto de datos ordenado

pais	anio	casos	poblacion
Afganistán	1999	745	19987071
Afganistán	2000	2666	20595360
Brasil	1999	37737	172006362
Brasil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

variables

pais	anio	casos	poblacion
Afganistán	1999	745	19987071
Afganistán	2000	2666	20595360
Brasil	1999	37737	172006362
Brasil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

observaciones

pais	anio	casos	poblacion
Afganistán	1999	745	19987071
Afganistán	2000	2666	20595360
Brasil	1999	37737	172006362
Brasil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

valores

Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



Conjunto de datos ordenado

- 1. Coloca cada conjunto de datos en un tibble.
- 2. Coloca cada variable en una columna.



Tabla 1

Pais	Año	Casos	Poblacion
Afganistan	1999	745	19987071
Afganistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Tabla 2

Pais	Año	Tipo	Cuenta
Afganistan	1999	Poblacion	19987071
Afganistan	2000	Poblacion	20595360
Afganistan	1999	Casos	745
Afganistan	2000	Casos	2666
Brazil	1999	Poblacion	172006362
Brazil	2000	Poblacion	174504898
Brazil	1999	Casos	37737
Brazil	2000	Casos	80488
China	1999	Poblacion	1272915272
China	2000	Poblacion	1280428583
China	1999	Casos	212258
China	2000	Casos	213766



Tabla 3

Pais	Año	Razón
Afganistan	1999	745/19987071
Afganistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583



Tabla 4

Pais	1999	2000
Afganistan	745	2666
Brazil	37737	80488
China	212258	213766

Pais	1999	2000
Afganistan	19987071	20595360
Brazil	172006362	174504898
China	1272915272	1280428583



Ilustración realizada por Allison Horst. Fuente: <https://blog.rstudio.com/2019/11/18/artist-in-residence/>



Gather() -> pivot_longer()

- La función gather toma múltiples columnas y las une en pares clave-valor.
- Esto permite resolver las situaciones en que tenemos columnas que realmente no representan variables, sino valores de una variable.



Gather() -> pivot_longer()

pais	anio	casos	pais	1999	2000
Afganistán	1999	745	Afganistán	745	2666
Afganistán	2000	2666	Brasil	37737	80488
Brasil	1999	37737	China	212258	213766
Brasil	2000	80488			
China	1999	212258			
China	2000	213766			

Tabla 4

Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



```
sandwich %>%  
  pivot_longer()
```



Spread() -> pivot_wider()



- Extender (**spread**) es lo opuesto de **gather**.
- Se usa cuando una observación aparece en múltiples filas.



Spread() -> pivot_wider()

país	año	tipo	casos
Afganistán	1999	casos	745
Afganistán	1999	población	19987071
Afganistán	2000	casos	2666
Afganistán	2000	población	20595360
Brasil	1999	casos	37737
Brasil	1999	población	172006362
Brasil	2000	casos	80488
Brasil	2000	población	174504898
China	1999	casos	212258
China	1999	población	1272915272
China	2000	casos	213766
China	2000	población	1280428583

Tabla 2

Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



wide

id	x	y	z
1	a	c	e
2	b	d	f

long

id	key	val
1	x	a
2	x	b
1	y	c
2	y	d
1	z	e
2	z	f

Fuente: <https://github.com/gadenbuie/tidyexplain>



wide

id	x	y	z
1	a	c	e
2	b	d	f

Fuente: <https://github.com/gadenbuie/tidyexplain>



Separate()

- Separa una columna en varias columnas, dividiendo de acuerdo a la posición de un carácter separador.



Separate()

pais	anio	tasa	pais	anio	casos	poblacion
Afganistán	1999	745 / 19987071	Afganistán	1999	745	19987071
Afganistán	2000	2666 / 20595360	Afganistán	2000	2666	20595360
Brasil	1999	37737 / 172006362	Brasil	1999	37737	172006362
Brasil	2000	80488 / 17504898	Brasil	2000	80488	17504898
China	1999	212258 / 1272915272	China	1999	212258	1272915272
China	2000	213766 / 1280428583	China	2000	213766	1280428583

Tabla 3

Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media



Unite()

- Es el inverso de separate()
- Combina múltiples columnas en una sola columna



Unite()

país	año	tasa
Afganistán	1999	745 / 19987071
Afganistán	2000	2666 / 20595360
Brasil	1999	37737 / 172006362
Brasil	2000	80488 / 17504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

país	siglo	casos	tasa
Afganistán	19	99	745 / 19987071
Afganistán	20	0	2666 / 20595360
Brasil	19	99	37737 / 172006362
Brasil	20	0	80488 / 17504898
China	19	99	212258 / 1272915272
China	20	0	213766 / 1280428583

Tabla 6

Fuente: Versión en español del libro R para Ciencia de Datos. Golemund, G. & Wickham, H. 2017. O'Reilly Media

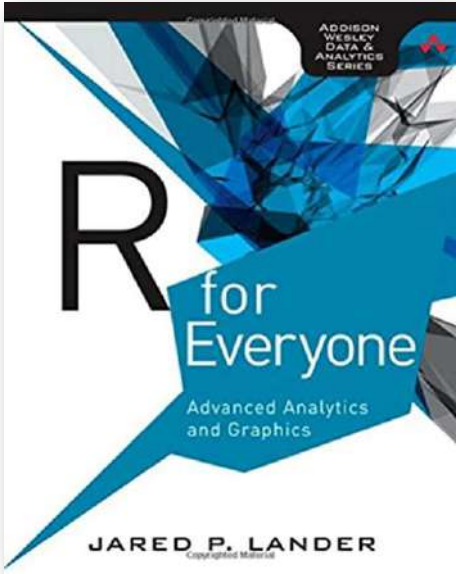
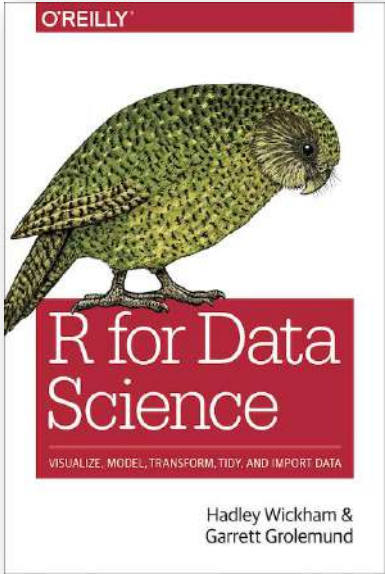
Recursos para seguir aprendiendo



Mg. Jesús Salinas Flores



jsalinas@lamolina.edu.pe



ESTADÍSTICA PARA TODOS